

CSE46901: Introduction to Robotics

Robot Operating System Development Environment Introduction

Professor Jaeyeon Lee

TA: Minhyuk Park

Preface

Welcome to CSE46901 Introduction to Robotics! This course is designed to help you get into the field of robot programming. As such, we need you to set up a development environment for the Robot Operating System. This document will help you to set up a development environment of our specification so that you can be productive with your time while taking this course.

Rationale behind the environment selection (optional)

Before we begin, we would like to address some questions you might have and our rationale for insisting on the setup we suggest to you. We believe this transparent approach will help you make your own decisions so that you can set up a productive environment for the requirements of this project. Also, knowing some behind the scenes decisions of this course might help you when you are doing a robotics project on your own outside this course.

There are many ways to program a robot. However, our focus in this course is to abstract the hardware aspect of a robot as much as possible so that you do not need a deep background in the mechanical and electrical aspect of a robot's operation. Also, we want you to be able to use your existing background in computer science to achieve the goal of programming a robot. As such, we have decided to go with the Robot Operating System(ROS) ecosystem.

Largely speaking, ROS consists of two main versions and a few subversions (distributions) with codenames. The two versions consist of ROS 1 and ROS 2. ROS 2 is an exciting new step forward for ROS, and improves on much of the old code in ROS 1. This does not mean that ROS1 is End of Life, however; active developments are still happening for ROS1. Also, ROS2 is very new and third party support for ROS2 is lacking as of 2022. Therefore, we opted to go for ROS1 to take advantage of the existing ecosystem.

Choosing a distribution for ROS is based on your host Operating System and third party program support. ROS is a meta-operating system installed on top of your Operating System like any other software. Therefore, choice of distribution depends mostly on your host operating system and third party ROS programs you want to

use. ROS officially supports Ubuntu Operating system as its host operating system, thus we opted to choose the latest Ubuntu Operating System with Long Term Support and the distribution that goes with it, Noetic Ninjemys. If you are using operating systems such as Debian, you may have to compile ROS from source, along with all the third party programs that we need you to use in this course. If you insist on using an unsupported version of Ubuntu and its ROS distribution, you need to change all references to “noetic” distribution in your install process into the distribution on your computer. Because we do not want to encourage you to go this route, we shall not share detailed workarounds we did to test our course materials on other platforms.

In summary, We will not support unsupported ROS configurations in this course. However, we will not penalize you if you manage to get it working for our course projects and produce expected outputs.

We also need to discuss why we insisted a native x86-64 computer for this project. ROS can be installed in ARM computers like Raspberry Pi and virtual machines and docker containers, so should you go with a native system? There are two reasons: network and graphics support. First of all, we need you to run visualization software like rqt and simulation software called gazebo in your environment. Softwares like rqt is normally not installed in a stripped down version of ROS meant for ARM processors. Even if you override that and install a full install version, you can experience lag from bloaty installation on your device. This is unacceptable for softwares like simulators that have to run in real time. Also, gazebo can show unexpected behavior such as blackscreen in virtualization environment with weak graphics support. You can also experience this problem if you are trying to get Windows Subsystem for Linux to display Graphics User Interface with hacks such as using Remote Desktop software to display the desktop of an WSL instance. Secondly, because ROS is a meta operating system, it is sensitive to network configurations even when it is not connected to remote computers. You might have problems of this kind if you are trying to use solutions such as docker or virtualization. Problems from this might need extensive knowledge in ROS to solve, and it is generally not worth your time, since solutions tend to be sensitive to small changes. In a nutshell, unless you are an expert in ROS capable of troubleshooting issues one by one, save your time and prepare a native x86-64 computer.

We will not support virtualized environments or non-x86-64 computers for this course for the ‘Remote PC’. Nevertheless, we will not penalize you if you manage to get it working and produce meaningful outputs.

How to get better in using Robot Operating System

This course is not an algorithmic course. There are specific objectives we need you to achieve with the robot we are providing you, simulated or physical, and we are not interested in efficiency or implementation details. Your source code is going to be checked for plagiarism, but that is about it. Therefore, your main objective should be twofold: learning how to interface with other code in a ROS standard way, and to plan actions in appropriate sequence and timing given constraints of the robot.

There are two main programming languages available for coding in Robot Operating Systems: C++ and Python. Either language is fine for this course, but we strongly recommend you to do your projects in python. Python's REPL (i.e. you can run python script with a flag such as "python -i mycode.py" and it will continue running after main has stopped. This will be useful to test parts of your code such as your function to move the robotic arm) is a powerful tool in robot debugging. Nevertheless, tutorials we are leaving you with will address both cases.

For more information on ROS itself, this free book from the robot's manufacturer can be a secondary textbook for you on Robot Operating System. Official: <https://www.robotis.us/ros-robot-programming-book-digital-copy/>
Mirror: https://ai.unist.ac.kr/~chiu/robot/books/ROS_Robot_Programming_EN.pdf

Alternatively, following wiki's instructions will be helpful to you. Start from "Navigating the ROS Filesystem" after installing ROS based on the instructions we will provide you below.

<http://wiki.ros.org/ROS/Tutorials>

We expect you to have foundational knowledge in Object Oriented Programming. If your coding foundation is weak, do not hesitate to grab a good python/ c++ book to revise on basics and Object Oriented Programming on the respective languages. Try to code example publishers and subscribers and try to read other people's code to get a feel of how to think in ROS mindset. Lastly, communicate and ask questions amongst your team members.

For your information, we prefer to just use vim to code ros code for both c++ and python. This is because it is convenient to use simple text editors and manage everything on a ssh terminal instead of using an integrated development environment (IDE) when you are managing multiple computers at the same time for a single robot. However, feel free to explore other options.

Have fun during the course!

Environment setup (Part 1 from Simulated Milestone 1)

We need you to prepare Ubuntu 20 PC for course milestones and the final project. From now onwards it will be called 'Remote PC'. It will provide you with a simulated environment and necessary visualization tools you need to interface with both the simulated and the physical robot. It is preferable for that PC to be portable, because you might want to use it for future milestones with a physical turtlebot, should we begin physical classes in the future. However, it is not necessary. We expect PCs of around 6th generation Intel i5 CPU with integrated graphics to be sufficient for this course.

The following instructions are based on <https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/#pc-setup>
Above manual might help you if you are not comfortable with our provided instruction.

We are using the ROS 1 version Noetic Ninjemys. Please select Noetic in the above website.

If we are to change to offline classes in the future, we will provide you with an Intel NUC portable computer with programs already installed along with physical turtlebot3. If you are using the provided NUC as the 'Remote PC', all programs you need are already installed.

Preparing ROS environment for the 'Remote PC'

1) Install Ubuntu 20.04.2.0 LTS (Focal Fossa, 64bit, Desktop)

- The installation file is `ubuntu-20.04.2.0-desktop-amd64.iso`
- hostname: `remote-pc`
- username: `YOUR_USER_NAME`
- password: `YOUR_PASSWORD`

2) Install some software on remote-pc:

```
sudo apt -y install vim
sudo apt -y install net-tools
sudo apt -y install openssh-server
sudo apt -y install curl
```

- You may want to remap the shortcut keys of Copy and Paste of your terminal.

3) Setup Network:

- Run `ifconfig` to see the IP of remote-pc. Remember the IP as `IP_OF_REMOTE_PC`.

4) Update Ubuntu software:

```
sudo apt-get update
sudo apt-get upgrade
```

5) Install ROS 1 Noetic Ninjemys (full version):

- Please refer to <http://wiki.ros.org/noetic/Installation/Ubuntu>

```
sudo sh -c 'echo "deb http://packages.ros.org/ros/ubuntu $(lsb_release -sc)
main" > /etc/apt/sources.list.d/ros-latest.list'
```

```
sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key
C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```

```
curl -sSL  
'http://keyserver.ubuntu.com/pks/lookup?op=get&search=0xC1CF6E31E6BADE8868  
B172B4F42ED6FBAB17C654' | sudo apt-key add -
```

```
sudo apt update
```

```
sudo apt install ros-noetic-desktop-full
```

```
echo "source /opt/ros/noetic/setup.bash" >> ~/.bashrc
```

```
source ~/.bashrc
```

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator  
python3-wstool build-essential
```

```
sudo rosdep init
```

```
rosdep update
```

6) Install TurtleBot 3 ROS 1 Packages:

- Please refer to

<https://emanual.robotis.com/docs/en/platform/turtlebot3/quick-start/#pc-setup>

```
sudo apt update
```

```
sudo apt upgrade
```

```
wget
```

```
https://raw.githubusercontent.com/ROBOTIS-GIT/robotis\_tools/master/install\_ros\_noetic.sh
```

```
chmod 755 ./install_ros_noetic.sh
```

```
bash ./install_ros_noetic.sh
```

- Press enter to continue the installation

```
sudo apt-get install ros-noetic-joy ros-noetic-teleop-twist-joy \  
ros-noetic-teleop-twist-keyboard ros-noetic-laser-proc \  
ros-noetic-rgbd-launch ros-noetic-rosserial-arduino \  
ros-noetic-rosserial-python ros-noetic-rosserial-client \  
ros-noetic-rosserial-msgs ros-noetic-amcl ros-noetic-map-server \  
ros-noetic-move-base ros-noetic-urdf ros-noetic-xacro \  
ros-noetic-compressed-image-transport ros-noetic-rqt* ros-noetic-rviz \  
ros-noetic-gmapping ros-noetic-navigation ros-noetic-interactive-markers
```

```
sudo apt update
sudo apt upgrade
sudo apt autoremove
```

7) Create workspace and download turtlebot3 packages

```
sudo apt install ros-noetic-dynamixel-sdk
sudo apt install ros-noetic-turtlebot3-msgs
sudo apt install ros-noetic-turtlebot3
```

- At this point, you can login as a different (non-root) user if you want to avoid using the account with sudo power.

- If ~/catkin_ws does not exist, run the following commands:

```
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws
catkin_make
```

8) Set ROS_MASTER_URI and ROS_HOSTNAME in .bashrc:

- Use "ifconfig" to find the IP address of the remote pc

- Make sure the following is added to ~/.bashrc:

```
source /opt/ros/noetic/setup.bash
source ~/catkin_ws/devel/setup.bash
export ROS_MASTER_URI=http://IP_OF_REMOTE_PC:11311
export ROS_HOSTNAME=IP_OF_REMOTE_PC
```

- Source .bashrc

```
source ~/.bashrc
```

9) Download turtlebot3 package:

```
cd ~/catkin_ws/src/
git clone https://github.com/ROBOTIS-GIT/turtlebot3_msgs.git
git clone https://github.com/ROBOTIS-GIT/turtlebot3.git
cd ~/catkin_ws && catkin_make
```

10) Open ~/.bashrc and add the following to the end:

```
export TURTLEBOT3_MODEL=waffle_pi
```

- Then source ~/.bashrc

```
source ~/.bashrc
```